

Natural Language Processing – Exercise 5

Sentiment Analysis with Neural Models

Due: July 5, 2026, 23:59

General Notes

1. To make things clearer and easier, we provide a structured API for this exercise along with helper classes. These are meant to help you, so whenever you feel limited by the provided API, feel free to deviate from it and change any of the given functions and classes (but be sure to explain your changes in the README).
2. You may use any PyTorch modules in your implementation.
3. For your submission, please provide your code (including `data_loader.py` if changed), a README file, and a PDF with all results and answers. Your code and README should be in a ZIP file; the PDF should be a separate file (not in the ZIP).

1 The Dataset

The dataset we will use is the **Stanford Sentiment Treebank**. It consists of sentences taken from movie reviews, each annotated with a sentiment value (a float between 0 – most negative – and 1 – most positive). We address a *binary* classification task: predicting 0 for negative sentiment and 1 for positive sentiment.

For training, a sentiment value is considered *positive* if ≥ 0.6 and *negative* if ≤ 0.4 ; examples with sentiment between 0.4 and 0.6 are discarded (this is implemented for you).

Sentences in the dataset are presented as trees, where each sub-tree defines a sub-phrase with its own sentiment value. We will train all models using the full sentences as well as the sub-sentences.

The dataset is provided as `stanfordSentimentTreebank.zip` in the exercise materials. Before running any code, extract it so that a folder named `stanfordSentimentTreebank` exists in your working directory.

The file `data_loader.py` handles all interaction with the dataset, including loading sentences (and sub-sentences) with binary labels, and splitting the data into train, validation, and test sets. Please read through it carefully.

Data splitting: We use an 80%/10%/10% split for train/validation/test. The train set is used to optimize model parameters; the validation set is used to select hyperparameters; the test set is used only for final unbiased evaluation.

Special test subsets: To better understand each model’s advantages and disadvantages, we also provide two special subsets extracted from the test set:

1. **Negated polarity** – Sentences where the sentiment polarity of one of the main sub-phrases is *opposite* to the whole sentence’s polarity (i.e., negation is in effect). Use `get_negated_polarity_examples` to retrieve the indices of these examples.
2. **Rare words** – We rank all test sentences by the maximum train-frequency of a sentiment word (a word labeled with non-neutral sentiment) and take the 50 sentences with the lowest value. Use `get_rare_words_examples` to retrieve the indices.

2 DataManager & OnlineDataset

We provide two utility classes:

DataManager handles:

1. Loading the dataset sentences with their labels.
2. Creating **OnlineDataset** instances (following the PyTorch **Dataset** API), using functions you will implement to convert token lists into valid model inputs.
3. Creating PyTorch **DataLoader** objects for iterating over batches as tensors.
4. Optionally subsampling the training set to a fixed size via the `max_train_samples` argument (a random subset is drawn with a fixed seed for reproducibility).

Please review both the class implementation and the PyTorch **DataLoader**/**Dataset** APIs.

3 Model Descriptions

You will implement **four** models:

Simple Log-Linear Model

This model uses a one-hot embedding for the words. The input is the **average** over all one-hot embeddings of the words in the sentence. After computing the average embedding, the model applies a single linear layer (a learned affine transformation) followed by a sigmoid to predict $p(y = \text{positive} \mid x)$.

Transformer-Embedding Log-Linear Model

This model is similar to the simple log-linear model, but instead of one-hot embeddings it uses the **embedding layer of a pre-trained Transformer** (described in Section 9).

Specifically, you will extract the token embedding matrix from the pre-trained model and use it as a fixed (frozen) look-up table. The input is the **average** over all Transformer token embeddings of the words in the sentence.

After computing the average embedding, the model applies a single linear layer followed by a sigmoid to predict $p(y = \text{positive} \mid x)$.

LSTM Model

The LSTM model is a more sophisticated architecture using a **bidirectional LSTM**. Each LSTM cell receives as input the Transformer token embedding of a word (the same frozen embeddings used in the log-linear model). We take the final hidden states of both directions of the bi-LSTM and concatenate them, then pass this concatenation through a linear layer and apply a sigmoid to produce $p(y = \text{positive} \mid x)$.

The hidden and cell state dimension of the LSTM will be **100**.

Fine-Tuned Transformer Model

The fourth model fine-tunes the full **distilroberta-base** Transformer end-to-end for the classification task. Unlike the previous two models which use the Transformer only as a static embedding source, here all model weights (including the Transformer backbone) are updated during training. A single linear classification head is placed on top of the final hidden state of the special [CLS] token, followed by a sigmoid to produce $p(y = \text{positive} \mid x)$. Training is implemented in native PyTorch (no HuggingFace Trainer).

The figure below illustrates the bi-LSTM architecture (with W representing the frozen Transformer embedding matrix):

4 Loss Function

We use **binary cross-entropy loss**. Since our model outputs a probability via a sigmoid, it is numerically preferable to combine the sigmoid with the loss computation. PyTorch provides `BCEWithLogitsLoss`, which accepts *logits* (i.e., values before the sigmoid) as input. Accordingly, the `forward` methods of your models should return logits (without applying the sigmoid), and `predict` should apply the sigmoid.

We optimize using the **Adam** optimizer.

5 Accuracy Measure

Accuracy is simply the fraction of correctly classified sentences. Since all models output a value in $[0, 1]$ (after sigmoid), we round to 0 or 1 for evaluation. Note: we do *not* discard values between 0.4 and 0.6 here.

1. Implement the function `binary_accuracy(preds, y)`, which computes the accuracy of predictions `preds` against true labels `y`.

6 Building the Simple Log-Linear Model

1. Complete the implementation of the `LogLinear` class. It will serve both the one-hot and the Transformer-embedding log-linear models. The `forward` method should return logits (no sigmoid); the `predict` method should apply the sigmoid.
2. Complete the `get_word_to_ind(word_count)` method, which creates a mapping from each vocabulary word to a unique integer index.
3. Complete the following helper methods:
 - a. `get_one_hot(size, ind)` – returns a one-hot numpy vector of length `size` with a 1 at position `ind`.
 - b. `average_one_hots(sent, word_to_ind)` – returns the average one-hot embedding for a sentence.
4. Implement the following training methods:
 - a. `train_epoch(model, data_iterator, optimizer, criterion)` – runs a single training epoch.
 - b. `evaluate(model, data_iterator, criterion)` – evaluates the model without updating parameters (use `torch.no_grad()`).
 - c. `train_model(model, data_manager, n_epochs, lr, weight_decay)` – runs the full training loop, recording train/validation loss and accuracy per epoch.
 - d. `train_log_linear_with_one_hot()` – instantiates all required objects and runs training.
5. Implement `get_predictions_for_data(model, data_iter)`, which collects all model predictions over a data iterator.
6. Train this model with:
 - Learning rate: 0.01
 - Epochs: 10
 - Batch size: 64

Please provide:

- a. A plot showing train and validation **loss** vs. epoch (both on the same graph).
- b. A plot showing train and validation **accuracy** vs. epoch (both on the same graph).
- c. The final test accuracy and test loss.

- d. Accuracy on each of the two special subsets from Section 1.

7 Building the Transformer-Embedding Log-Linear Model

In this section you will implement the second log-linear model, which replaces one-hot vectors with embeddings from a pre-trained Transformer.

We will use the **distilroberta-base** model from the HuggingFace `transformers` library. You will load the model, extract its token embedding matrix (the `embeddings.word_embeddings` weight), and use it as a fixed look-up table – **do not fine-tune the embedding weights**.

1. Implement `load_transformer_embeddings()`, which loads `distilroberta-base` and returns the vocabulary size and the embedding weight matrix (as a numpy array or tensor).
2. Implement `get_transformer_average(sent, tokenizer, embedding_matrix, embedding_dim)`, which tokenizes a sentence, maps each token to its embedding (using the frozen matrix), and returns the average embedding vector. Tokens that appear in the tokenizer’s vocabulary but not in the embedding matrix should be treated as unknown (zero vector).
3. Implement `train_log_linear_with_transformer()`, which creates all required objects and runs the training process.
4. Follow the same training protocol as in Section 6 (same hyperparameters, plots, and reported values). That is: learning rate 0.01, no weight decay, 10 epochs, batch size 64.

8 Building the LSTM Model

1. Implement the `LSTM` class following the bi-directional LSTM architecture described in Section 3. Apply **dropout** regularization on the input to the final linear layer during training. Use PyTorch’s built-in `nn.LSTM` module.
2. The input embeddings are the same frozen Transformer embeddings used in Section 7. Each word in the sentence is first mapped to its Transformer token embedding, and the sequence of embeddings is fed into the bi-LSTM.

Because batches require all sequences to be the same length, all sentences are padded or truncated to **length 52**:

- Sentences with **more** than 52 tokens: keep the first 52 embeddings.
 - Sentences with **fewer** than 52 tokens: pad with zero-vectors to reach length 52.
 - Sentences with **exactly** 52 tokens: no change.
3. Implement `sentence_to_embedding(sent, word_to_vec, seq_len, embedding_dim)`, which maps a sentence to a fixed-size numpy array of shape `(seq_len, embedding_dim)`.

4. Implement `train_lstm_with_transformer()`, which creates all required objects and runs the training process.
5. Train the model with:
 - Learning rate: 0.001
 - Weight decay: 0.0001
 - Dropout probability: 0.5
 - Epochs: 4
 - Batch size: 64

Provide the same plots and reported values as in Section 6.

9 Fine-Tuning a Transformer Model

In this part, you will **fine-tune** the full **distilroberta-base** Transformer for sentiment classification using **native PyTorch** (no HuggingFace Trainer).

You will use the *base* model only (no added classification head from the library); instead, you will add your own linear classification layer on top of the [CLS] token's final hidden state (the first token output).

1. Implement the `TransformerSentimentClassifier` class:
 - Load **distilroberta-base** (using `AutoModel.from_pretrained`).
 - Add a linear layer mapping from the model's hidden dimension to 1 output (for binary classification).
 - The **forward** method should return logits (no sigmoid).
 - Tokenize inputs using `AutoTokenizer.from_pretrained("distilroberta-base")` with truncation and padding.
2. Implement `train_transformer()`, training with:
 - Learning rate: 10^{-5}
 - Weight decay: 0
 - Epochs: 2
 - Batch size: 64
 - Training set size: **2500 randomly sampled sentences** (use `DataManager`'s `max_train_samples` argument; do not use sub-phrases).

Use `BCEWithLogitsLoss` and the Adam optimizer. Training must be implemented in native PyTorch (no Trainer).

3. Provide the same plots and reported values as in Section 6.

10 Comparing the Models

1. Compare the test and validation accuracy of the **simple log-linear model** and the **Transformer-embedding log-linear model**. Which performs better? Provide a possible explanation.
2. Compare those results with the **LSTM** and **fine-tuned Transformer** models. Which perform better? Explain. Note that the fine-tuned Transformer was trained on a much smaller training set (2500 sentences) than the other models – make sure to address this in your discussion.
3. Compare all four models on the **two special subsets**. For each subset, state which model achieves the highest (and lowest) accuracy, and provide a possible explanation.

11 Theoretical Question: Counting Parameters

Consider an encoder that looks up each input token in an embedding matrix and then processes the resulting vectors. Let d be the model dimension, V the vocabulary size, n the context length, and assume the encoder has L layers. Ignore any classification head, and to keep things simple ignore all biases.

For the Transformer, count only (i) the token embedding matrix and (ii) the self-attention projections, using a single head and ignoring positional embeddings, the feed-forward sub-layer, and LayerNorm. So each Transformer layer has just the four projections W_Q, W_K, W_V, W_O , each of shape $d \times d$.

1. **Transformer.** Write the number of parameters in (a) the token embedding matrix and (b) the whole encoder (embedding + the L attention layers). For a realistic vocabulary (say $V = 30,000$, $d = 512$), which part dominates? Does any part depend on the context length n ?
2. **RNN.** Now replace the attention layers by L recurrent layers, where one layer's (time-shared) recurrent transformation has a fixed number of parameters R (you do not need to expand it). Write the total number of parameters. Does it depend on n ?
3. **Characters as tokens.** Suppose we tokenize by characters instead of words: the vocabulary shrinks a lot (say ~ 100 instead of $\sim 30,000$), but to cover texts of the same length the context length n grows (e.g. by $\sim 5\times$). For each model, write the new parameter counts: (a) by what factor does the size of the embedding matrix ($V \cdot d$) shrink, and (b) do the attention projections or the recurrent parameters change at all when V shrinks and n grows? Combining (a) and (b), does each model's total number of parameters go up or down, and why?

12 Tips & Recommendations

1. Read the PyTorch documentation carefully for each layer or function you use – especially input/output shapes and types.
2. Watch for dimension and type mismatches. Be careful when reshaping tensors not to accidentally shuffle values.
3. Use `with torch.no_grad():` during evaluation to avoid computing unnecessary gradients.
4. The Transformer model will be slow to train on CPU; use a GPU if available (`get_available_device()` is provided).
5. When loading the Transformer embedding matrix for Sections 7–8, save it locally after the first load (the provided `load_transformer_embeddings()` already does this). That way you avoid re-downloading it on every run.
6. The frozen embedding matrix in Sections 7–8 should be a plain tensor (or numpy array) used for look-up, not a trainable `nn.Embedding` layer.